DX Full Stack Development Guide

Volume 3 - End-to-End Standard

React Front-End + Node MVC Backend + Microsoft SQL Server

Bagian dari DNIA DX Development Starter Kit. Panduan ini menyatukan Front-End Template dan Backend Template menjadi satu gambaran end-to-end: bagaimana request mengalir, bagaimana keamanan dijaga di kedua sisi, dan bagaimana fitur baru dibangun secara konsisten.

> **Pesan utama**
> Front-End dan Backend memakai filosofi yang sama: separation of concern, security by design, testing by default, dan architecture governance. Pahami satu alur penuh, lalu Anda bisa mengembangkan fitur di kedua sisi dengan pola yang sama.

Audience: Semua DX Intern (Front-End, Back-End, Full Stack)
Document owner: PED / Digital Transformation
Classification: Internal / Proprietary

1. Gambaran Umum Starter Kit

DNIA DX Development Starter Kit terdiri dari tiga volume yang saling melengkapi.

Volume	Fokus	Untuk Siapa
Vol 1 - Backend Template Guide	Node 22, Express 5, Sequelize 6, SQL Server	Back-End dan Full Stack Intern
Vol 2 - Front-End Template Guide	React 19, TypeScript, Vite 8, Tailwind 4	Front-End dan Full Stack Intern
Vol 3 - Full Stack Development Guide	Integrasi end-to-end kedua template	Semua DX Intern

2. Arsitektur End-to-End

```
User
  -> React Front-End (View)
  -> Feature Controller (hooks / orchestration)
  -> TanStack Query
  -> Axios (withCredentials + signed CSRF)
  ==================  HTTP / REST  ==================
  -> Node MVC Backend (Route -> Middleware -> Validation)
  -> Controller
  -> Service (business logic)
  -> Model (Sequelize)
  -> Microsoft SQL Server
```

> **Simetri arsitektur**
> Front-End memakai MVC per fitur (model, controller, view). Backend memakai MVC berlapis (controller, service, model). Keduanya memisahkan UI/HTTP, logika, dan akses data.

3. Pemetaan Layer Front-End dan Backend

Front-End	Backend	Tanggung Jawab Bersama
view	controller	Batas antarmuka (UI atau HTTP).
controller (hooks)	service	Orchestration dan business logic.
model (API/schema/type/state)	model (Sequelize)	Kontrak data dan akses data.
shared/lib (api, csrf)	middleware (auth, csrf)	Keamanan dan cross-cutting concern.

Front-End	Backend	Tanggung Jawab Bersama
Zod (form)	Zod (request schema)	Validasi input berlapis.

4. Alur Autentikasi End-to-End

Tahap	Yang Terjadi
1. Login	View LoginPage mengirim kredensial melalui controller dan Axios (withCredentials).
2. Backend memverifikasi	Route auth memvalidasi, service memeriksa user, password dicek dengan bcrypt.
3. Set cookie	Backend mengirim JWT pada HttpOnly cookie; browser tidak menyimpan JWT di localStorage.
4. Protected route	ProtectedRoute front-end mengizinkan akses; request berikutnya membawa cookie otomatis.
5. CSRF	Untuk request mutasi, interceptor menambahkan signed CSRF token yang divalidasi middleware backend.

5. Alur Data Work Order (Contoh)

```
WorkOrdersPage (view)
  -> useWorkOrders (controller / hook)
  -> TanStack Query fetch
  -> Axios GET /api/v1/work-orders
  -> Backend route -> validation -> controller -> service -> model
  -> SQL Server (APP_T_WORK_ORDER)
  -> response -> cache -> render table
```

Untuk create dan update, alur yang sama berlaku dengan method POST atau PATCH, ditambah signed CSRF token dan pemeriksaan role (Operator atau Admin) di backend.

6. Model Keamanan Bersama

Aspek	Front-End	Backend
Token	JWT hanya di HttpOnly cookie	Menerbitkan dan memverifikasi JWT
CSRF	Interceptor menambah signed token	Middleware memvalidasi token

Aspek	Front-End	Backend
Validation	Zod pada form	Zod strict schema pada request
XSS	Plain text, tanpa dangerouslySetInnerHTML	Sanitization dan output aman
Transport	withCredentials + HTTPS	CORS allowlist, Helmet, rate limit
Authorization	UI menyesuaikan role	Role check: Viewer, Operator, Admin

Prinsip penting
Backend selalu menjadi security boundary. Validasi dan kontrol di front-end meningkatkan UX, tetapi keputusan keamanan final ada di server.

7. Quality Gate Bersama

Tahap	Front-End	Backend
Lint	npm run lint (ESLint)	npm run lint (ESLint)
Test	npm test (Vitest)	npm test (node:test)
Build / Architecture	npm run build (tsc + Vite)	npm run check:architecture
Aggregate	npm run check	npm run check
Security	npm audit	npm audit
Static analysis	npm run sonar	npm run sonar

Kedua template memakai gerbang kualitas yang sama sehingga standar handover konsisten di seluruh project DX.

8. Menambah Fitur Baru End-to-End

Gunakan urutan berikut untuk membangun satu fitur dari database hingga UI.

Langkah	Backend	Front-End
1. Database	Tambah tabel/kolom via SQL script terkontrol	-
2. Model	Mapping Sequelize	Definisikan type dan schema Zod
3. Logic	Service untuk business logic	Controller/hook untuk orchestration

Langkah	Backend	Front-End
4. Interface	Route dan controller HTTP	View: page dan komponen
5. Keamanan	Middleware auth, role, CSRF	withCredentials, CSRF interceptor
6. Test	node:test	Vitest + Testing Library
7. Quality	npm run check, audit, sonar	npm run check, audit, sonar

9. Roadmap Onboarding 2 Minggu

Rekomendasi roadmap, bukan requirement eksplisit repository.

Minggu 1 - Memahami

- Setup Front-End dan Backend Template secara lokal.
- Import Postman collection, buat token, dan coba /auth/me.
- Login melalui UI dan amati alur ProtectedRoute.
- Telusuri satu request dari view front-end hingga SQL Server.
- Baca README, CONTRIBUTING, dan SECURITY kedua template.

Minggu 2 - Membangun

- Buat satu fitur kecil end-to-end (model, controller, view di kedua sisi).
- Tambahkan satu endpoint backend dan konsumsi dari front-end.
- Tambahkan table, filter, atau export pada UI.
- Tulis test di front-end dan backend.
- Jalankan npm run check, npm audit, dan npm run sonar di kedua project.

10. Deployment End-to-End

Komponen	Yang Perlu Disiapkan
Backend	Schema SQL Server dibuat via script, environment variable diisi, health check GET /health hijau.
Front-End	VITE_API_BASE_URL diarahkan ke endpoint API, build menghasilkan dist/.
Networking	Origin front-end didaftarkan pada CORS backend; HTTPS aktif di kedua sisi.
Serving	dist/ disajikan dengan SPA fallback, CSP sesuai, dan cache immutable untuk asset ber-hash.
Security	Secret berbeda per environment, rotasi bila ada indikasi bocor, tidak commit credential.

11. Prinsip yang Harus Selalu Dipegang

- Pertahankan pemisahan layer di front-end dan backend.
- Jangan pernah menyimpan data autentikasi di localStorage.
- Selalu lewati validasi dan sanitization; backend adalah security boundary.
- Jangan menambahkan raw SQL di source backend, jangan memanggil sequelize.sync().
- Reuse komponen dan library shared, hindari duplikasi.
- Jangan melewati quality gate untuk mempercepat penyelesaian.

> **Tujuan akhir**
> Setiap batch intern baru memulai dari standar arsitektur yang sama, sehingga seluruh aplikasi DX konsisten, aman, dan mudah dipelihara.

12. Referensi

Panduan ini menyatukan sumber dari kedua template:

- Front-End: README.md, package.json, app/App.tsx, app/router.tsx, config/env.ts, features (auth, work-orders), shared/lib (api, csrf, queryClient, utils).
- Backend: README.md, CONTRIBUTING.md, SECURITY.md, package.json, scripts (check-runtime, check-architecture, verify-database-schema, seed-demo), database/001_create_schema.sql, postman/DX-Template.postman_collection.json, dan folder test.